

Session-1

Task 1: What is a "Program"?

A **program** is a set of step-by-step instructions written in a language a computer can understand, telling it exactly what to do to complete a task. Just like a recipe tells a cook what ingredients to use and in what order, a program tells a computer what data to use and what operations to perform on it.

Real-life example: Instagram is a program (actually a huge collection of programs) that tells your phone how to display photos in a feed, how to play videos when you tap them, how to send a "like" to the server when you double-tap a picture, and how to notify you when someone comments. Every action you take in the app triggers a piece of code (a small program) that was written by a developer to handle exactly that action.

Task 2: Three Programming Languages

Language Compiled / Interpreted

Java Both — Java code is first *compiled* into "bytecode," which is then *interpreted* (or JIT-compiled) by the Java Virtual Machine (JVM) at runtime.

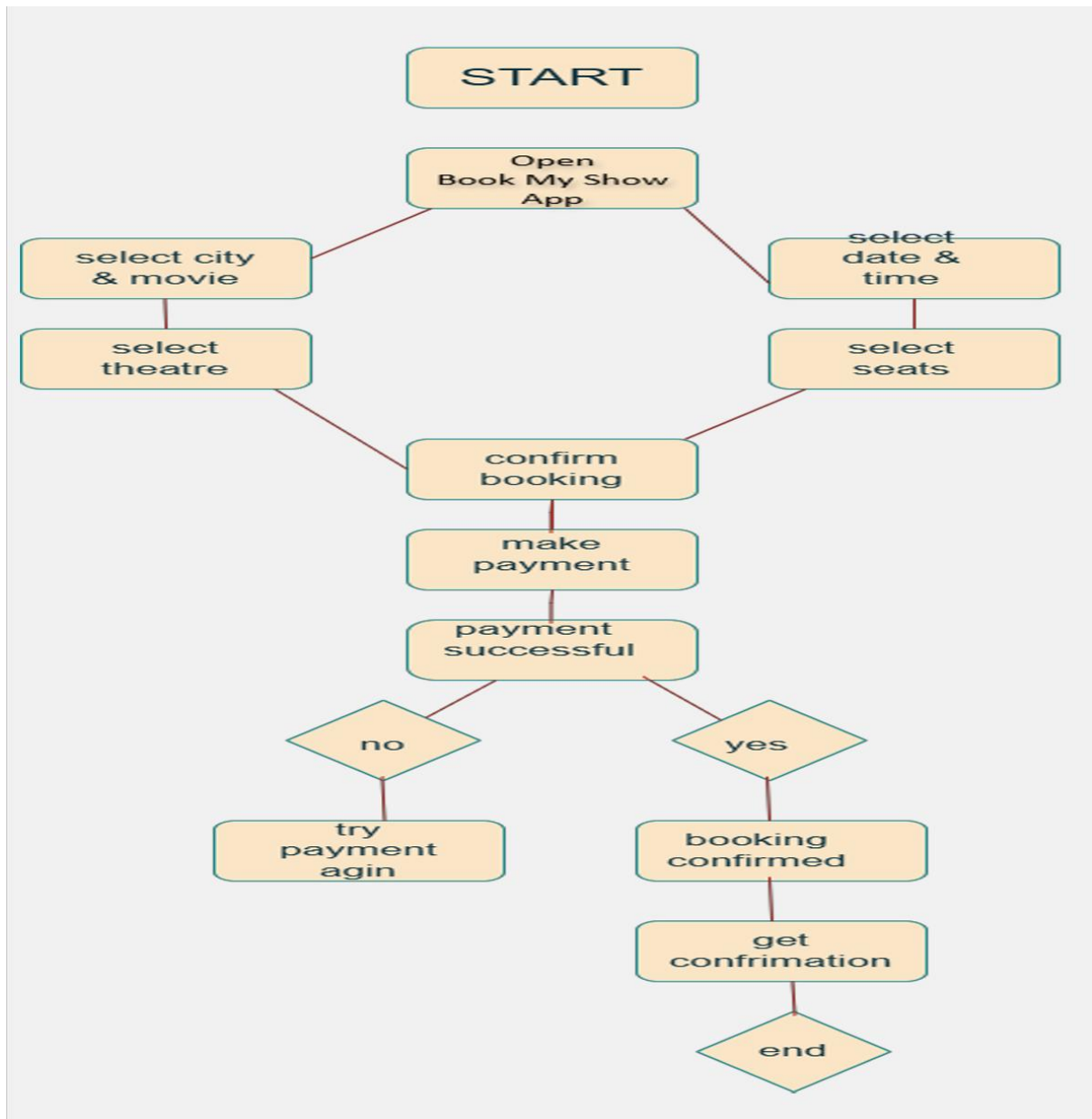
Python Interpreted — Python code runs line-by-line through the Python interpreter, though it does compile to bytecode internally first.

JavaScript Interpreted (traditionally) — runs inside a browser's JavaScript engine line-by-line, though modern engines like V8 use JIT compilation for speed.

*(For contrast, C and C++ are classic examples of purely **compiled** languages — the whole program is translated into machine code before it ever runs.)*

Task 3: Flowchart — Booking a Movie Ticket on BookMyShow

Here's a text-based flowchart you can redraw with boxes and arrows on paper or in an online tool (draw.io, Lucidchart, Canva):



Task 4: Algorithm — Ordering Food on Swiggy

1. Open the Swiggy app on the phone.
2. Check/confirm the delivery location (GPS or manually entered address).
3. Browse restaurants or search for a specific dish/restaurant.
4. Select a restaurant from the list.
5. Browse the menu and select desired food items.
6. Add selected items to the cart.
7. Review the cart (check items, quantity, and price).

8. If changes are needed, edit the cart (add/remove items) and repeat step 7.
9. Proceed to checkout.
10. Confirm/select the delivery address.
11. Choose a payment method (UPI, card, wallet, cash on delivery, etc.).
12. Apply a coupon code, if available.
13. Confirm the order and complete payment.
14. Receive order confirmation with estimated delivery time.
15. Track the order status until it is delivered.
16. **End** — Order received.

Session-2

Task 1: Install GCC and Verify

Windows:

1. Download **MinGW-w64** (or use the easier route: install **MSYS2** from [msys2.org](https://www.msys2.org), then run `pacman -S mingw-w64-ucrt-x86_64-gcc` in the MSYS2 terminal).
2. Add the bin folder (e.g., `C:\msys64\ucrt64\bin`) to your **System PATH** environment variable.
3. Open Command Prompt and check installation.

Mac:

`xcode-select --install`

This installs Apple's Clang, which works as gcc on Mac.

Linux (Ubuntu/Debian):

`bash`

`sudo apt update`

`sudo apt install build-essential`

Verify on any OS, open a terminal and run:

`bash`

`gcc --version`

You should see output like:

`gcc (MinGW-W64) 13.2.0`

Copyright (C) 2023 Free Software Foundation, Inc.

(Exact version/text varies by OS — what matters is that it prints a version instead of "command not found.")

Task 2: Create hello.c

Open VS Code (or DevC++) → File → New File → save it as hello.c. Type this in:

```
#include <stdio.h>

int main() {

    printf("Hello, Instagram World!\n");

    return 0;

}
```

Task 3: Same Code with Labels

```
c

#include <stdio.h>    // Header: gives access to printf() for input/output

int main() {         // Main function: program execution starts here

    printf("Hello, Instagram World!\n"); // Statement: prints text to console

    return 0;        // Return statement: tells OS the program ended successfully
    (0 = no error)

}
```

Quick explanation of each part:

- **Header (#include <stdio.h>)** — imports the Standard Input/Output library so you can use functions like printf() and scanf().
- **main() function** — every C program must have exactly one main() function; it's the entry point where execution begins.
- **return 0;** — sends a status code back to the operating system. 0 conventionally means "program finished successfully"; a non-zero value usually signals an error.

Task 4: Compile and Run

In the terminal, navigate to the folder containing hello.c, then run:

```
bash
```

```
gcc hello.c -o hello
```

This tells GCC: *compile hello.c and output an executable named hello.*

Then run it:

OS	Command
----	---------

Windows	hello.exe or just hello
---------	-------------------------

Linux/Mac	./hello
-----------	---------

Expected output:

Hello, Instagram World!

Common issues to watch for:

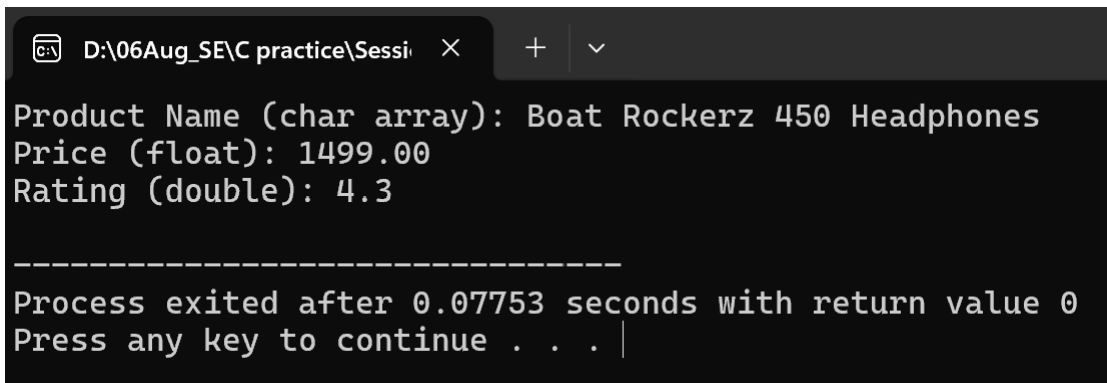
- gcc: command not found → GCC isn't installed or not added to PATH; recheck Task 1.
- permission denied on Linux/Mac when running ./hello → run chmod +x hello first.
- Missing semicolon or unmatched braces → C is strict about syntax; the compiler error message will point to the line number.

Session-3

Task 1: Flipkart Product Variables

```
1  #include <stdio.h>
2  int main() {
3      char productName[50] = "Boat Rockerz 450 Headphones";
4      float price = 1499.00f;
5      double rating = 4.3;
6      printf("Product Name (char array): %s\n", productName);
7      printf("Price (float): %.2f\n", price);
8      printf("Rating (double): %.1f\n", rating);
9      return 0;
10 }
```

Output:



```
Product Name (char array): Boat Rockerz 450 Headphones
Price (float): 1499.00
Rating (double): 4.3

-----
Process exited after 0.07753 seconds with return value 0
Press any key to continue . . .
```

Task

2: GST Constant — Zomato Order

```
1  #include <stdio.h>
2  int main() {
3      const float GST_RATE = 0.18f;
4      float basePrice = 500.0f;
5      float gstAmount = basePrice * GST_RATE;
6      float finalPrice = basePrice + gstAmount;
7      printf("Base Price: Rs. %.2f\n", basePrice);
8      printf("GST (18%): Rs. %.2f\n", gstAmount);
9      printf("Final Price: Rs. %.2f\n", finalPrice);
10     return 0;
11 }
```

Output:

```
D:\06Aug_SE\C practice\Sessi X + v

Base Price: Rs. 500.00
GST (18%): Rs. 90.00
Final Price: Rs. 590.00

-----
Process exited after 0.09834 seconds with return value 0
Press any key to continue . . . |
```

Task 3: Spotify Playlist Info

```
1 #include <stdio.h>
2 int main() {
3     char playlistName[50] = "Late Night Vibes";
4     int totalSongs = 42;
5     float avgDuration = 3.5f; // in minutes
6     printf("My playlist \"%s\" has %d songs, with an average duration of %.1f minutes per song.\n",
7           playlistName, totalSongs, avgDuration);
8     return 0;
9 }
```

Output:

```
D:\06Aug_SE\C practice\Sessi X + v

My playlist "Late Night Vibes" has 42 songs, with an average duration of 3.5 minutes per song.

-----
Process exited after 0.1272 seconds with return value 0
Press any key to continue . . . |
```

Task 4: Invalid Variable Names — Corrected

Original	Valid?	Reason	Corrected
1stPlayer	 Invalid	Variable names cannot start with a digit	firstPlayer or player1st
player_age	 Valid	Letters, digits, underscore — starts with a letter	player_age (no change needed)
\$score	 Invalid	\$ is not an allowed character in C identifiers (only letters, digits, underscore)	score or dollarScore

Original	Valid?	Reason	Corrected
total-marks	 Invalid	Hyphen - is not allowed — it gets interpreted as a minus/subtraction operator	total_marks or totalMarks
userName	 Valid	Starts with a letter, uses valid characters (camelCase)	userName (no change needed)

General C variable naming rules:

1. Must start with a **letter** (a–z, A–Z) or an underscore _ — never a digit.
2. Can only contain **letters, digits, and underscores** — no spaces or special characters like \$, -, @, etc.
3. Cannot be a **reserved keyword** (e.g., int, float, return).
4. C is **case-sensitive** — Score and score are treated as different variables.
5. No spaces allowed — use camelCase (totalMarks) or snake_case (total_marks) instead.

Session-4

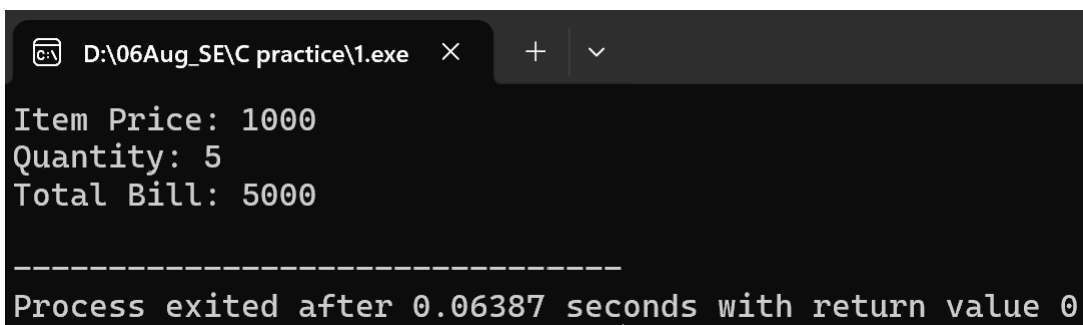
Operators and Expressions Topics: Arithmetic, Logical, Relational, Increment/Decrement. **Operator precedence.** Example: Simple calculator for addition, subtraction, etc. **Pending**

Tasks

1. Create a simple JavaScript function called `calculateTotal` that takes two numbers: `itemPrice` and `quantity`, and returns the total bill amount using arithmetic operators

```
1  #include <stdio.h>
2  int calculateTotal (int itemPrice, int quantity){
3      return itemPrice * quantity;
4  }
5  int main(){
6      int itemPrice = 1000;
7      int quantity = 5;
8      int totalBill;
9      totalBill = calculateTotal(itemPrice, quantity);
10     printf("Item Price: %d\n", itemPrice);
11     printf("Quantity: %d\n", quantity);
12     printf("Total Bill: %d\n", totalBill);
13     return 0;
14 }
```

Output:



```
D:\06Aug_SE\C practice\1.exe  X  +  v

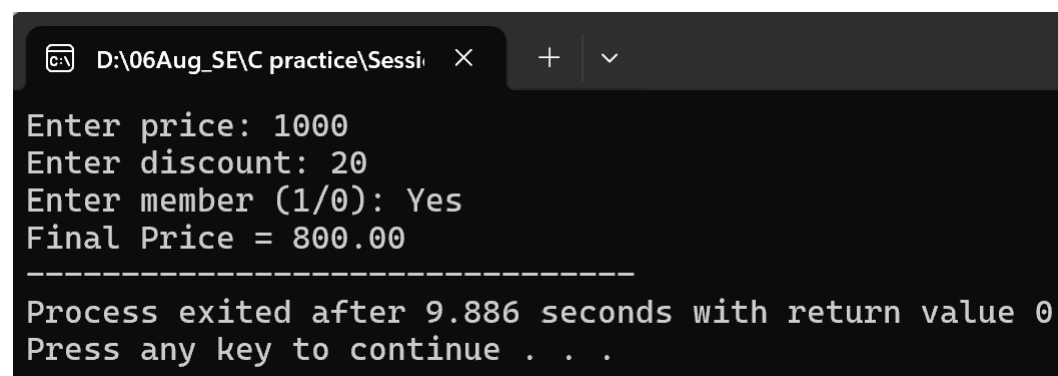
Item Price: 1000
Quantity: 5
Total Bill: 5000

-----
Process exited after 0.06387 seconds with return value 0
```

2. Build a Flipkart-style discount calculator: given product price, discount percentage, and a boolean isMember, use arithmetic and logical operators to calculate the final price (apply an extra 5% off if isMember is true)

```
1  #include <stdio.h>
2  int main(){
3      float price, discount, final;
4      int member;
5      printf("Enter price: ");
6      scanf("%f", &price);
7      printf("Enter discount: ");
8      scanf("%f", &discount);
9      printf("Enter member (1/0): ");
10     scanf("%d", &member);
11     final = price - (price * discount / 100);
12     if (member == 1)
13         final = final - (final * 5 / 100);
14     printf("Final Price = %.2f", final);
15     return 0;
16 }
```

Output:



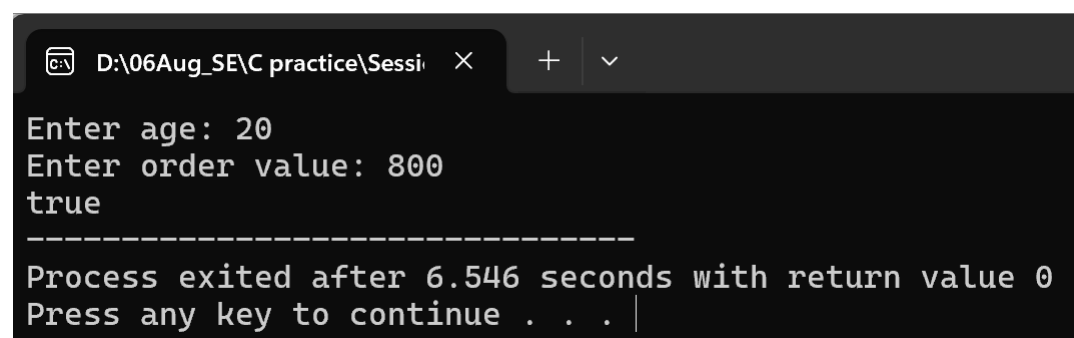
```
D:\06Aug_SE\C practice\Sessi  x  +  v
Enter price: 1000
Enter discount: 20
Enter member (1/0): Yes
Final Price = 800.00
-----
Process exited after 9.886 seconds with return value 0
Press any key to continue . . .
```

3. Write a function `isEligibleForOffer` that takes a user's age and total order value, and returns true if the user is 18 or older AND the order value is above 500, otherwise false.

Hint: Use relational and logical operators together.

```
1  #include <stdio.h>
2  int main(){
3      int age;
4      float orderValue;
5      printf("Enter age: ");
6      scanf("%d", &age);
7      printf("Enter order value: ");
8      scanf("%f", &orderValue);
9      if (age >= 18 && orderValue > 500)
10         printf("true");
11     else
12         printf("false");
13     return 0;
14 }
```

Output:

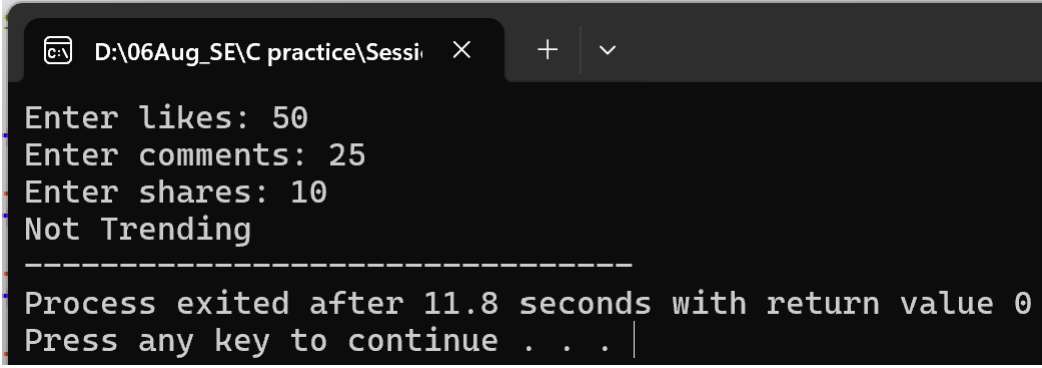


```
D:\06Aug_SE\C practice\Sessi  X  +  v
Enter age: 20
Enter order value: 800
true
-----
Process exited after 6.546 seconds with return value 0
Press any key to continue . . . |
```

4. Given three variables: likes, comments, and shares (all numbers), write code to check if a post is 'trending' on Instagram (at least 1000 likes OR more than 200 comments AND at least 50 shares). Print the result.

```
1  #include <stdio.h>
2  int main() {
3      int likes, comments, shares;
4      printf("Enter likes: ");
5      scanf("%d", &likes);
6      printf("Enter comments: ");
7      scanf("%d", &comments);
8      printf("Enter shares: ");
9      scanf("%d", &shares);
10     if (likes >= 1000 || (comments > 200 && shares >= 50)) {
11         printf("Trending");
12     }
13     else {
14         printf("Not Trending");
15     }
16     return 0;
17 }
```

Output:

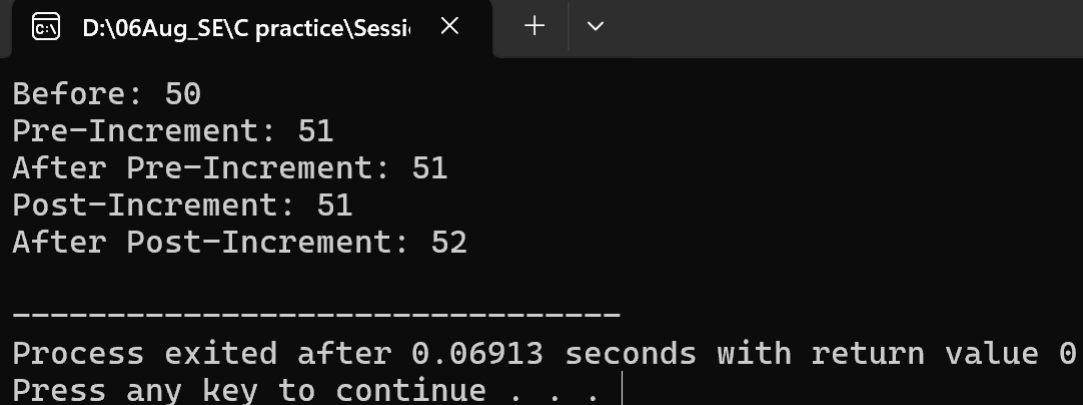


```
D:\06Aug_SE\C practice\Sessi  X  +  v
Enter likes: 50
Enter comments: 25
Enter shares: 10
Not Trending
-----
Process exited after 11.8 seconds with return value 0
Press any key to continue . . . |
```

5. Write a code snippet that demonstrates the difference between pre-increment (++count) and post-increment (count++) by logging the values before and after using both on a followerCount variable

```
1  #include <stdio.h>
2  int main()
3  {
4      int followerCount = 50;
5
6      printf("Before: %d\n", followerCount);
7
8      printf("Pre-Increment: %d\n", ++followerCount);
9      printf("After Pre-Increment: %d\n", followerCount);
10
11     printf("Post-Increment: %d\n", followerCount++);
12     printf("After Post-Increment: %d\n", followerCount);
13
14     return 0;
15 }
```

Output:



```
D:\06Aug_SE\C practice\Sessi x + v
Before: 50
Pre-Increment: 51
After Pre-Increment: 51
Post-Increment: 51
After Post-Increment: 52

-----
Process exited after 0.06913 seconds with return value 0
Press any key to continue . . . |
```

Session-5

Conditional Statements Topics: If, Else If Ladder, Nested If, Switch Case. **Example:** “Study Mood Bot” – takes user’s input and prints motivation message.

Tasks

1. Create a simple IPL Fan Bot that takes your favorite IPL team name as input and uses if-else-if statements to print a unique cheer message for each team (e.g., 'Go Mumbai Indians!', 'Chennai Super Kings for the win!'). If the team is not recognized, print 'Team not found!'

```
1  #include <stdio.h>
2  #include <string.h>
3  int main(){
4      char team[20];
5      printf("Enter IPL team: ");
6      scanf("%s", team);
7      if (strcmp(team, "mumbai") == 0)
8      {
9          printf("Go Mumbai Indians!");
10     }
11     else if (strcmp(team, "gujarat") == 0)
12     {
13         printf("Go Gujarat Titans!");
14     }
15     else if (strcmp(team, "kolkata") == 0)
16     {
17         printf("Go Kolkata Knight Riders!");
18     }
19     else if (strcmp(team, "rajasthan") == 0)
20     {
21         printf("Go Rajasthan Royals!");
22     }
23     else
24     {
25         printf("Team not found!");
26     }
27     return 0;
28 }
```

Output:

```
D:\06Aug_SE\C practice\Sessi X + v
Enter IPL team: mumbai
Go Mumbai Indians!
-----
Process exited after 4.277 seconds with return value 0
Press any key to continue . . .
```

2. Build a Zomato-style food suggestion tool: take the user's preferred meal time ('breakfast', 'lunch', 'dinner', or 'snack') and use a switch-case statement to suggest a popular dish for that time. If the input doesn't match any meal, suggest 'Try some fruits!'.

```
1  #include <stdio.h>
2  int main(){
3      int choice;
4      printf("1. Breakfast\n");
5      printf("2. Lunch\n");
6      printf("3. Dinner\n");
7      printf("4. Snack\n");
8      printf("Enter your choice: ");
9      scanf("%d", &choice);
10     switch(choice){
11         case 1:
12             printf("Try Dosa!");
13             break;
14         case 2:
15             printf("Try Gujarati Thali!");
16             break;
17         case 3:
18             printf("Try Pizza!");
19             break;
20         case 4:
21             printf("Try Samosa!");
22             break;
23         default:
24             printf("Try some fruits!");
25             return 0;
26     }
27     return 0;
28 }
```

Output:

```
D:\06Aug_SE\C practice\Sessi X + v
1. Breakfast
2. Lunch
3. Dinner
4. Snack
Enter your choice: 2
Try Gujarati Thali!
-----
Process exited after 5.321 seconds with return value 0
Press any key to continue . . . |
```

3. Create a Flipkart discount calculator that asks the user for the total cart amount. Use nested if statements to check: if amount > 2000, apply 20% discount; else if amount > 1000, apply 10% discount; else, no discount. Print the final amount to pay.

Hint: Use nested ifs to check each discount slab.

```
1  #include <stdio.h>
2  int main(){
3      float amount, discount, finalAmount;
4      printf("Enter cart amount: ");
5      scanf("%f", &amount);
6      if (amount > 2000){
7          discount = amount * 20 / 100;
8      }
9      else{
10         if(amount > 1000){
11             discount = amount * 10 / 100;
12         }
13         else{
14             discount = 0;
15         }
16     }
17     finalAmount = amount - discount;
18     printf("Discount = %.2f\n", discount);
19     printf("Final amount to pay = %.2f", finalAmount);
20     return 0;
21 }
```

Output:

```
D:\06Aug_SE\C practice\Sessi X + v
Enter cart amount: 2000
Discount = 200.00
Final amount to pay = 1800.00
-----
Process exited after 6.161 seconds with return value 0
Press any key to continue . . . |
```

4. Write a program that takes a user's age and checks eligibility for three things using if-else statements: if age ≥ 18 , print 'Eligible for Driving License'; if age ≥ 21 , also print 'Eligible for Credit Card'; if age ≥ 25 , also print 'Eligible for Car Rental'. Print all applicable messages for the given age.

```
1  #include <stdio.h>
2  int main(){
3      int age;
4      printf("Enter your age: ");
5      scanf("%d", &age);
6      if(age >= 18)
7          printf("Eligible for Driving License\n");
8      if (age >= 21)
9          printf("Eligible for Credit Card\n");
10     if(age >= 25)
11         printf("Eligible for Car Rental\n");
12     return 0;
13 }
```

Output:

```
D:\06Aug_SE\C practice\Sessi X + v
Enter your age: 20
Eligible for Driving License
-----
Process exited after 4.576 seconds with return value 0
Press any key to continue . . . |
```

Session-6

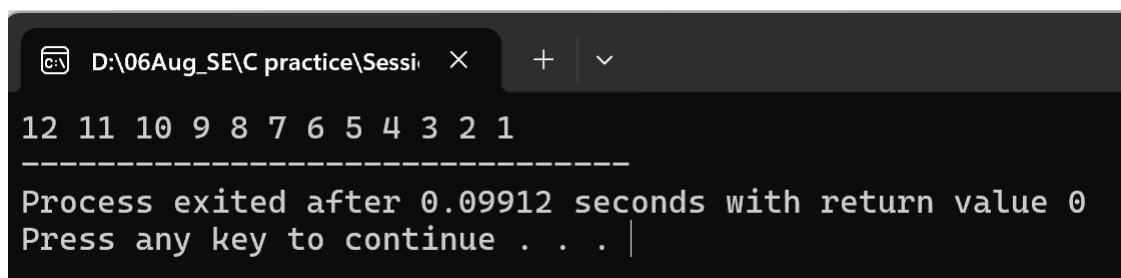
Loops (For, While, Do-While) Topics: Difference between entry-controlled and exit-controlled loops. Use cases (e.g., menu-driven programs). Example: Countdown timer using loops.

Tasks

1. Write a program that prints the numbers from 10 down to 1 using a for loop, similar to a countdown timer.

```
1  #include <stdio.h>
2  int main(){
3      for(int i= 12; i >= 1; i--){
4          printf("%d ", i);
5      }
6      return 0;
7  }
```

Output:



```
D:\06Aug_SE\C practice\Sessi  X  +  v
12 11 10 9 8 7 6 5 4 3 2 1
-----
Process exited after 0.09912 seconds with return value 0
Press any key to continue . . .
```

2. Create a menu-driven console app that lets the user: 1) View your favorite 3 IPL teams, 2) Add a new team, 3) Exit. Use a while loop to keep showing the menu until the user chooses Exit.

Hint: Use input() (or Scanner in Java) to get the user's choice each time.

```

1  #include <stdio.h>
2  int main()
3  {
4      int choice;
5      char team[20];
6      while(1)
7      {
8          printf("\n1. View IPL Teams");
9          printf("\n2. Add New Team");
10         printf("\n3. Exit");
11         printf("\nEnter your choice: ");
12         scanf("%d", &choice);
13         if(choice == 1)
14         {
15             printf("\nCSK");
16             printf("\nMI");
17             printf("\nRCB\n");
18         }
19         else if(choice == 2)
20         {
21             printf("\nEnter New Team: ");
22             scanf("%s", team);
23             printf("Team Added: %s\n", team);
24         }
25         else if(choice == 3)
26         {
27             printf("\nThank You!\n");
28             break;
29         }
30         else
31         {
32             printf("\nInvalid choice! Please try again.\n");
33         }
34     }
35     return 0;
36 }

```

Output:

```

D:\06Aug_SE\C practice\Sessi ×
1. View IPL Teams
2. Add New Team
3. Exit
Enter your choice: 1

CSK
MI
RCB

1. View IPL Teams
2. Add New Team
3. Exit
Enter your choice: 2

Enter New Team: KKR
Team Added: KKR

```

3. Build a 'Guess the Song' game like Spotify — the program randomly picks a song name from a list and asks the user to guess it. Use a do-while loop so the user can keep guessing until they get it right.

Constraint: Use at least 3 song names of your choice.

```
[*] struct3.cpp  struct4.cpp  1.cpp  [*] Session42.cpp  [*] Session43.cpp  [*] Session44.cpp  Session45.cpp  Session51.cpp  [*] Session52.cpp  [*] Session53.cpp
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5  #include <time.h>
6
7  int main()
8  {
9      char *songs[] = {"Shape of You", "Blinding Lights", "Levitating", "Believer", "Perfect"};
10     char *hints[] = {
11         "Ed Sheeran, released in 2017, about falling in love at first sight",
12         "The Weeknd, 80s-inspired synth-pop, released in 2019",
13         "Dua Lipa, disco-pop hit from 2020",
14         "Imagine Dragons, high-energy rock anthem",
15         "Ed Sheeran, a romantic wedding favorite"
16     };
17     int totalSongs = 5;
18     char guess[50];
19     int attempts;
20
21     srand(time(0));
22     int index = rand() % totalSongs;
23
24     printf("=== Guess the Song ===\n");
25     printf("Hint: %s\n\n", hints[index]);
26
27     attempts = 0;
28     do
29     {
30         printf("Enter your guess: ");
31         fgets(guess, sizeof(guess), stdin);
32
33         guess[strcspn(guess, "\n")] = '\0';
34
35         attempts++;
36
37         if (strcasecmp(guess, songs[index]) == 0)
38         {
39             printf("\nCorrect! The song was \"%s\". You guessed it in %d attempt(s).\n",
40                 songs[index], attempts);
41             break;
42         }
43         else
44         {
45             printf("Wrong guess, try again!\n\n");
46         }
47     } while (strcasecmp(guess, songs[index]) != 0);
48
49     return 0;
50 }
51 }
```

Output:

```
D:\06Aug_SE\C practice\Sessi x + v
=== Guess the Song ===
Hint: Dua Lipa, disco-pop hit from 2020

Enter your guess: 2
Wrong guess, try again!

Enter your guess: Levitating

Correct! The song was "Levitating". You guessed it in 2 attempt(s).

-----
Process exited after 11.34 seconds with return value 0
Press any key to continue . . . |
```

4. Explain with your own example the difference between entry-controlled and exit-controlled loops by writing a short code snippet for each (for/while vs do-while) and describing what happens if the loop condition is false at the start.

Entry-controlled

```
1  #include <stdio.h>
2  int main()
3  {
4      int count = 10;
5      while (count < 5)    // condition is FALSE right away
6      {
7          printf("This will print, count = %d\n", count);
8          count++;
9      }
10     printf("Loop finished. Final count = %d\n", count);
11     return 0;
12 }
```

Output:

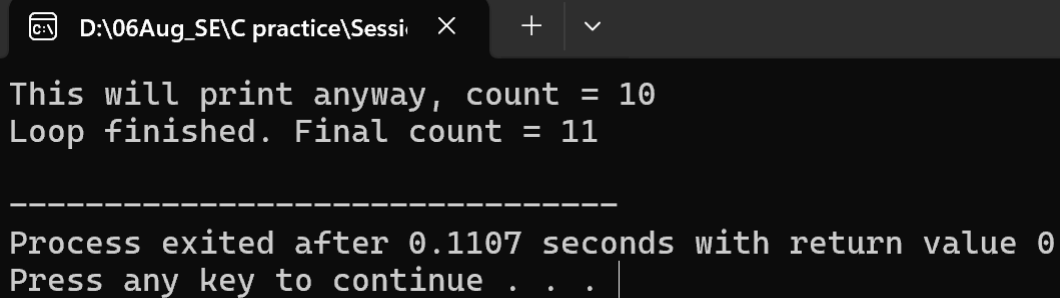
```
D:\06Aug_SE\C practice\Sessi x + v
Loop finished. Final count = 10

-----
Process exited after 0.09153 seconds with return value 0
Press any key to continue . . . |
```

Exit-controlled

```
1  #include <stdio.h>
2  int main()
3  {
4      int count = 10;
5      do
6      {
7          printf("This will print anyway, count = %d\n", count);
8          count++;
9      } while (count < 5); // condition checked AFTER the body runs
10     printf("Loop finished. Final count = %d\n", count);
11     return 0;
12 }
```

Output:



```
D:\06Aug_SE\C practice\Sessi  X  +  v

This will print anyway, count = 10
Loop finished. Final count = 11

-----
Process exited after 0.1107 seconds with return value 0
Press any key to continue . . . |
```

Session-7

Nested Loops & Pattern Printing Topics: Using nested loops for grids or triangle patterns. Example: Star pyramid pattern.

Tasks

1. Use nested for loops to print a grid of emojis representing a 5x5 Instagram post feed, where each cell shows a 📷 symbol.

```
1  #include <stdio.h>
2  int main(){
3      int i, j;
4      for(i=1; i<= 5; i++) {
5          for(j = 1; j <= 5; j++){
6              printf("#");
7          }
8          printf("\n");
9      }
10     return 0;
11 }
```

Output:

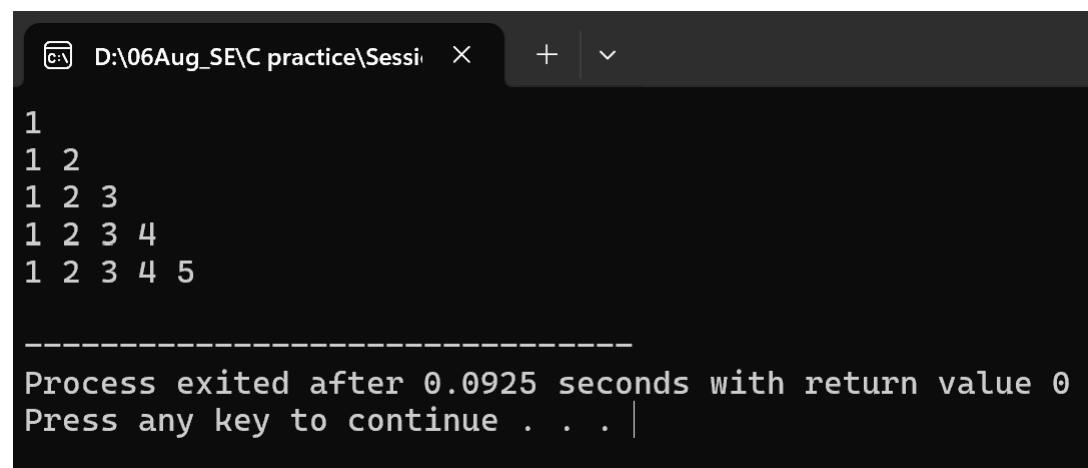
```
D:\06Aug_SE\C practice\Sessi  x  +  v
#####
#####
#####
#####
#####

-----
Process exited after 0.1279 seconds with return value 0
Press any key to continue . . . |
```

2. Build a right-angled triangle pattern using nested loops, where each row displays increasing numbers starting from 1, similar to how a leaderboard on a gaming app shows rank numbers.

```
1  #include <stdio.h>
2  int main(){
3      int j;
4      for(int i=1; i <= 5; i++){
5          for(j = 1; j <= i; j++){
6              printf("%d ", j);
7          }
8          printf("\n");
9      }
10     return 0;
11 }
```

Output:



```
D:\06Aug_SE\C practice\Sessi  x  +  v
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

-----
Process exited after 0.0925 seconds with return value 0
Press any key to continue . . .
```

3. Create a pattern that prints a pyramid of stars (*) with 6 rows, centered like the loading animation you see on BookMyShow when a page is loading.

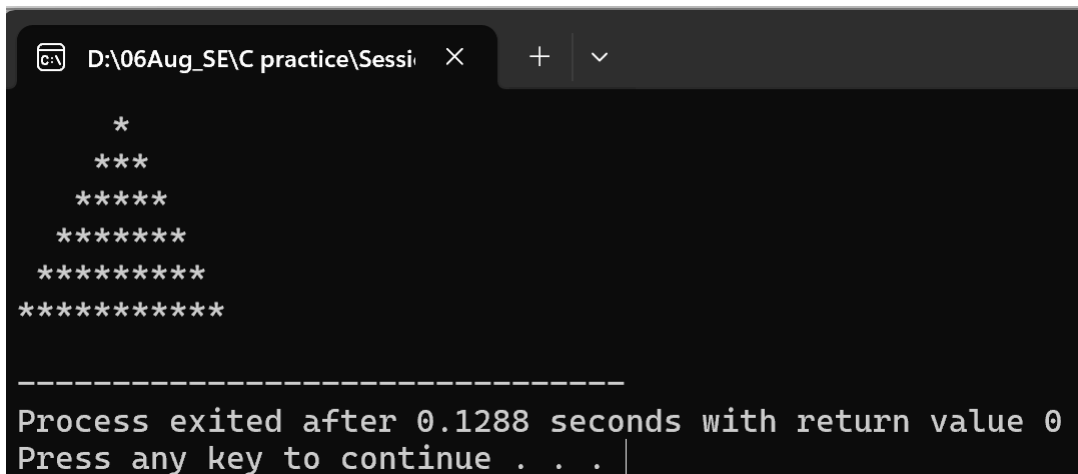
Hint: Use spaces to align the stars in the center for each row.

```

1  #include <stdio.h>
2  int main(){
3      int i, j, space;
4      for(i=1; i <= 6; i++){
5          for(space = 1; space <= 6 - i; space++){
6              printf(" ");
7          }
8          for(j = 1; j <= 2 * i - 1; j++){
9              printf("*");
10         }
11         printf("\n");
12     }
13     return 0;
14 }

```

Output:



```

      *
     ***
    *****
   ********
  *********
 *****
*****
-----
Process exited after 0.1288 seconds with return value 0
Press any key to continue . . .

```

4. Write code using nested loops to print a pattern of alternating 0s and 1s in a grid, like the checkered background seen in some Spotify playlist covers (e.g., for a 4x4 grid, alternate 0 and 1 in each cell).

```

1  #include <stdio.h>
2  int main(){
3      int i, j;
4      for(i = 1; i <= 4; i++)
5      {
6          for(j = 1; j <= 4; j++)
7          {
8              if((i + j) % 2 == 0)
9                  printf("0 ");
10             else
11                 printf("1 ");
12         }
13         printf("\n");
14     }
15     return 0;
16 }

```

Output:

```

D:\06Aug_SE\C practice\Sessi  ×  +  v
0 1 0 1
1 0 1 0
0 1 0 1
1 0 1 0

-----
Process exited after 0.06994 seconds with return value 0
Press any key to continue . . .

```

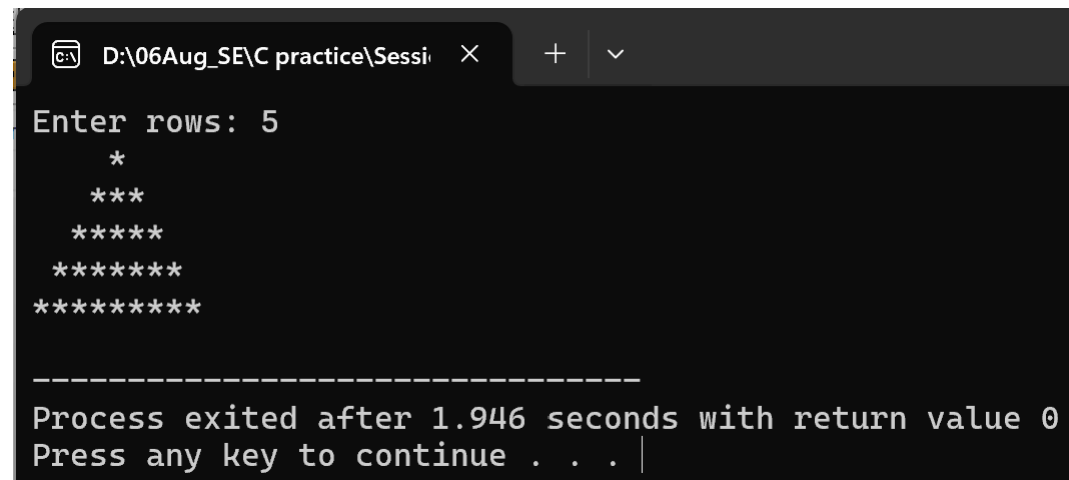
5. Modify your pyramid pattern code to accept the number of rows as user input, so the user can set the height of the pyramid before printing

```

1  #include <stdio.h>
2  int main(){
3      int i, j, space, n;
4      printf("Enter rows: ");
5      scanf("%d", &n);
6      for(i = 1; i <= n; i++)
7      {
8          for(space = 1; space <= n-i; space++)
9              printf(" ");
10         for(j = 1; j <= 2*i-1; j++)
11             printf("*");
12         printf("\n");
13     }
14     return 0;
15 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi
Enter rows: 5
 *
 ***
*****
*****
*****
-----
Process exited after 1.946 seconds with return value 0
Press any key to continue . . . |

```

Session-8

Functions Topics: Declaration, Definition, Calling. Pass-by-value vs pass-by-reference.

Tasks

1. Declare a function called get User Initials that takes a user's full name (like 'Virat Kohli') and returns their initials in uppercase (e.g., 'VK'). Call this function with your favourite cricketer's name and print the result

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  void getUserInitials(char name[], char initials[])
6  {
7      int i = 0, j = 0;
8      while (name[i] == ' ')
9          i++;
10
11     initials[j++] = toupper(name[i]);
12     while (name[i] != '\0')
13     {
14         if (name[i] == ' ' && name[i + 1] != '\0')
15         {
16             initials[j++] = toupper(name[i + 1]);
17             break;
18         }
19         i++;
20     }
21
22     initials[j] = '\0';
23 }
24
25 int main()
26 {
27     char name[] = "Virat Kohli";
28     char initials[10];
29
30     getUserInitials(name, initials);
31
32     printf("Name: %s\n", name);
33     printf("Initials: %s\n", initials);
34
35     return 0;
36 }
```

Output:

```
D:\06Aug_SE\C practice\Sessi X + v
Name: Virat Kohli
Initials: VK

-----
Process exited after 0.0854 seconds with return value 0
Press any key to continue . . . |
```

2. Create a function `addToCart` that takes a shopping cart array and a product name, adds the product to the cart, and prints the updated cart. Demonstrate how passing the cart array by reference allows changes to persist outside the function.

Hint: In languages like JavaScript, arrays are passed by reference. In C/C++, use pointers for reference behavior.

```
1  #include <stdio.h>
2  #include <string.h>
3  void addToCart(char cart[][50], int *count, char product[]){
4      strcpy(cart[*count], product);
5      (*count)++;
6
7      printf("\nUpdated Cart:\n");
8      for (int i = 0; i < *count; i++){
9          printf("%d. %s\n", i + 1, cart[i]);
10     }
11 }
12 int main(){
13     char cart[10][50];
14     int count = 0;
15     addToCart(cart, &count, "Laptop");
16     addToCart(cart, &count, "Mouse");
17     addToCart(cart, &count, "Keyboard");
18
19     printf("\nFinal Cart Outside Function:\n");
20     for (int i = 0; i < count; i++){
21         printf("%d. %s\n", i + 1, cart[i]);
22     }
23
24     return 0;
25 }
```

Output:

```
D:\06Aug_SE\C practice\Sessi X + v

Updated Cart:
1. Laptop

Updated Cart:
1. Laptop
2. Mouse

Updated Cart:
1. Laptop
2. Mouse
3. Keyboard

Final Cart Outside Function:
1. Laptop
2. Mouse
3. Keyboard

-----
Process exited after 0.08133 seconds with return value 0
Press any key to continue . . . |
```

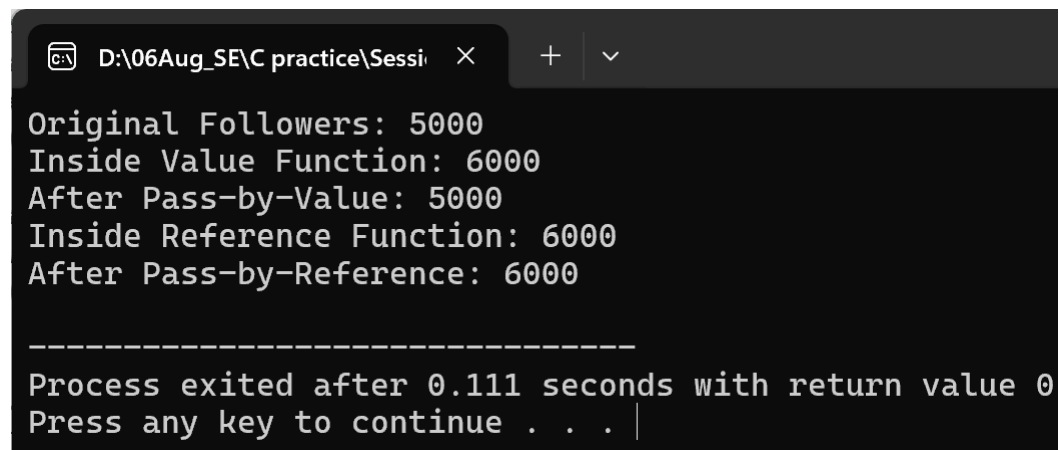
3. Write two functions: increase Followers By Value and increase Followers By Reference. Each should take a followers count and add 1000 to it, but one should use pass-by-value and the other pass-by-reference. Show how the original followers count changes (or doesn't) after calling each function.

```

1  #include <stdio.h>
2  void increaseFollowersByValue(int followers){
3      followers = followers + 1000;
4      printf("Inside Value Function: %d\n", followers);
5  }
6  void increaseFollowersByReference(int *followers){
7      *followers = *followers + 1000;
8      printf("Inside Reference Function: %d\n", *followers);
9  }
10
11 int main(){
12     int followers = 5000;
13     printf("Original Followers: %d\n", followers);
14     increaseFollowersByValue(followers);
15     printf("After Pass-by-Value: %d\n", followers);
16     increaseFollowersByReference(&followers);
17     printf("After Pass-by-Reference: %d\n", followers);
18     return 0;
19 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi  X  +  v
Original Followers: 5000
Inside Value Function: 6000
After Pass-by-Value: 5000
Inside Reference Function: 6000
After Pass-by-Reference: 6000

-----
Process exited after 0.111 seconds with return value 0
Press any key to continue . . . |

```

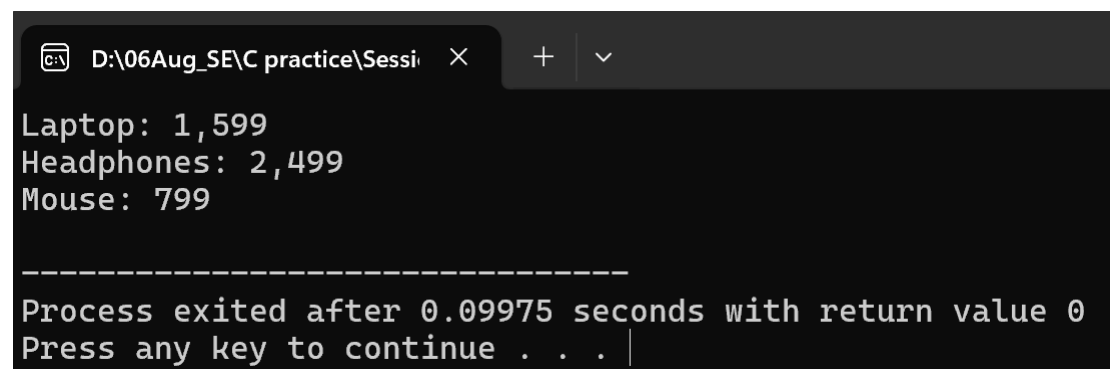
4. Build a function format Price that takes a price in rupees and returns a string formatted like Flipkart's price tag (e.g., '₹1,599'). Use this function to display prices for three different products

```

1  #include <stdio.h>
2  void formatPrice(int price, char result[]){
3      if (price >= 1000){
4          sprintf(result, "%d,%03d", price / 1000, price % 1000);
5      }
6      else{
7          sprintf(result, "%d", price);
8      }
9  }
10
11 int main(){
12     char price1[30];
13     char price2[30];
14     char price3[30];
15
16     formatPrice(1599, price1);
17     formatPrice(2499, price2);
18     formatPrice(799, price3);
19
20     printf("Laptop: %s\n", price1);
21     printf("Headphones: %s\n", price2);
22     printf("Mouse: %s\n", price3);
23
24     return 0;
25 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi
Laptop: 1,599
Headphones: 2,499
Mouse: 799

-----
Process exited after 0.09975 seconds with return value 0
Press any key to continue . . .

```

5. Refactor an existing function you wrote above to make it reusable for both product names and usernames (for example, a function that capitalizes the first letter of any string).

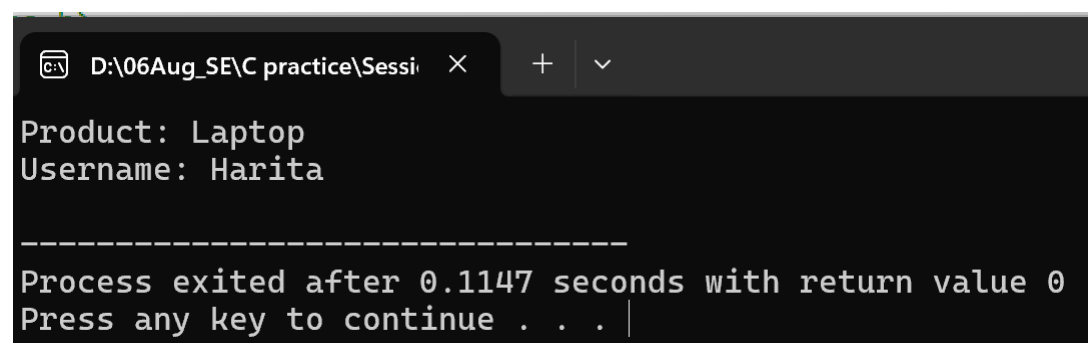
Constraint: The refactored function should work for any string input, not just a specific use case.

```

1  #include <stdio.h>
2  #include <ctype.h>
3
4  void capitalizeFirst(char str[])
5  {
6      if (str[0] != '\0')
7      {
8          str[0] = toupper(str[0]);
9      }
10 }
11
12 int main()
13 {
14     char product[] = "laptop";
15     char username[] = "harita";
16
17     capitalizeFirst(product);
18     capitalizeFirst(username);
19
20     printf("Product: %s\n", product);
21     printf("Username: %s\n", username);
22
23     return 0;
24 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi
Product: Laptop
Username: Harita

-----
Process exited after 0.1147 seconds with return value 0
Press any key to continue . . . |

```

Session-9

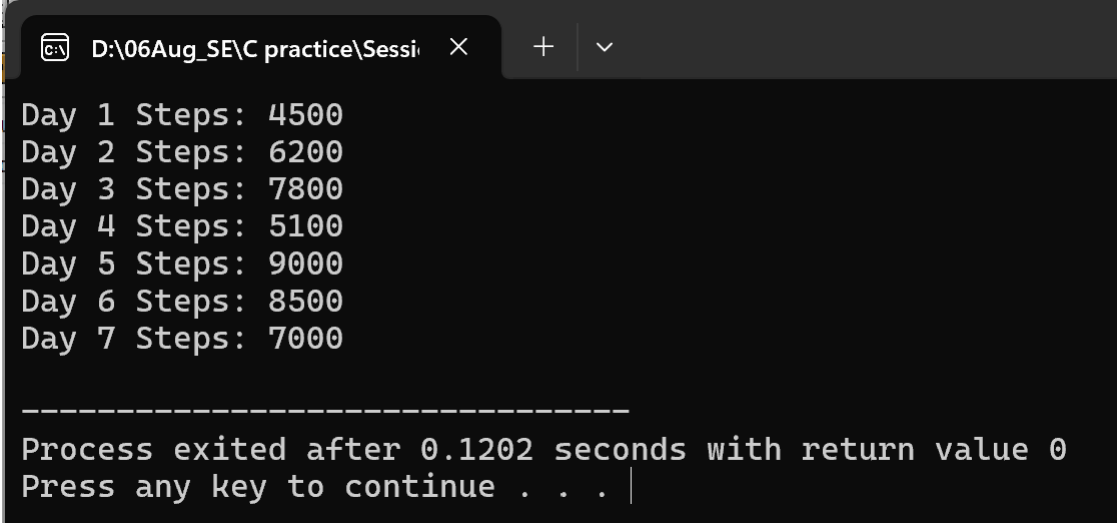
ArraysTopics:1D & 2D Arrays, Array Traversal . Example: Record 7-day screen time & calculate average.

Tasks

1. Declare a 1D array called `dailySteps` with 7 elements to store your step count for each day of the week, assign sample values, and print each value using a loop.

```
1  #include <stdio.h>
2  int main()
3  {
4      int dailySteps[7] = {4500, 6200, 7800, 5100, 9000, 8500, 7000};
5      for(int i = 0; i < 7; i++)
6      {
7          printf("Day %d Steps: %d\n", i + 1, dailySteps[i]);
8      }
9
10     return 0;
11 }
```

Output:



```
D:\06Aug_SE\C practice\Sessi  X  +  v
Day 1 Steps: 4500
Day 2 Steps: 6200
Day 3 Steps: 7800
Day 4 Steps: 5100
Day 5 Steps: 9000
Day 6 Steps: 8500
Day 7 Steps: 7000

-----
Process exited after 0.1202 seconds with return value 0
Press any key to continue . . . |
```

2. Create a 2D array called `playlistRatings` to store ratings for 3 Spotify playlists over 5 days (rows = playlists, columns = days). Fill it with sample numbers and print the ratings for the second playlist.

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int playlistRatings[3][5] =
6      {
7          {5, 4, 5, 3, 4},
8          {4, 5, 4, 5, 5},
9          {3, 4, 4, 5, 4}
10     };
11
12     printf("Ratings of Second Playlist:\n");
13
14     for(int i = 0; i < 5; i++)
15     {
16         printf("Day %d: %d\n", i + 1, playlistRatings[1][i]);
17     }
18
19     return 0;
20 }

```

Output:

```

D:\06Aug_SE\C practice\Sessi  ×  +  v
Ratings of Second Playlist:
Day 1: 4
Day 2: 5
Day 3: 4
Day 4: 5
Day 5: 5

-----
Process exited after 0.1173 seconds with return value 0
Press any key to continue . . . |

```

3. Build a function that takes a 1D array of 7 integers representing your daily Zomato order amounts and calculates the average spend for the week.

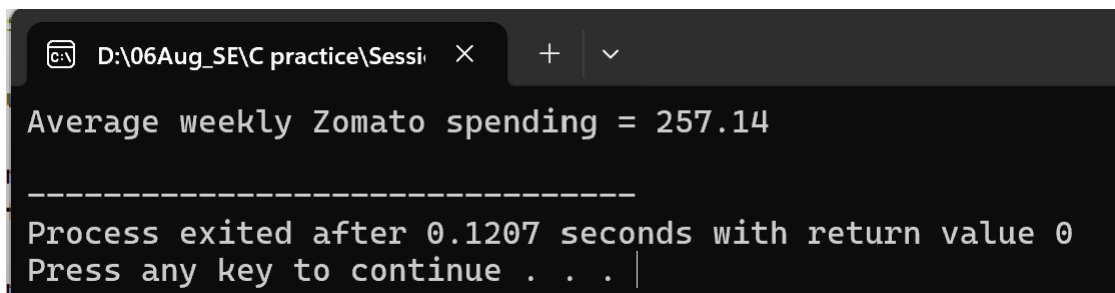
Hint: Use a loop to sum the values, then divide by the array length.

```

1  #include <stdio.h>
2
3  float calculateAverage(int orders[], int size)
4  {
5      int sum = 0;
6      for(int i = 0; i < size; i++)
7      {
8          sum = sum + orders[i];
9      }
10     return (float)sum / size;
11 }
12
13 int main()
14 {
15     int orders[7] = {250, 180, 320, 150, 400, 220, 280};
16     float average;
17     average = calculateAverage(orders, 7);
18     printf("Average weekly Zomato spending = %.2f\n", average);
19     return 0;
20 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi
Average weekly Zomato spending = 257.14
-----
Process exited after 0.1207 seconds with return value 0
Press any key to continue . . . |

```

4. Given a 2D array called `cricketScores` where each row represents an IPL match and columns represent runs scored by each team, write code to print the highest score from each match

```

1  #include <stdio.h>
2  int main(){
3      int cricketScores[4][2] =
4      {
5          {185, 172},
6          {201, 195},
7          {165, 180},
8          {210, 198}
9      };
10     for(int i = 0; i < 4; i++)
11     {
12         if(cricketScores[i][0] > cricketScores[i][1])
13         {
14             printf("Match %d Highest Score: %d\n",
15                 i + 1, cricketScores[i][0]);
16         }
17         else
18         {
19             printf("Match %d Highest Score: %d\n",
20                 i + 1, cricketScores[i][1]);
21         }
22     }
23
24     return 0;
25 }

```

Output:

```

D:\06Aug_SE\C practice\Sessi  ×  +  v
Match 1 Highest Score: 185
Match 2 Highest Score: 201
Match 3 Highest Score: 180
Match 4 Highest Score: 210

-----
Process exited after 0.1208 seconds with return value 0
Press any key to continue . . .

```

Session-10

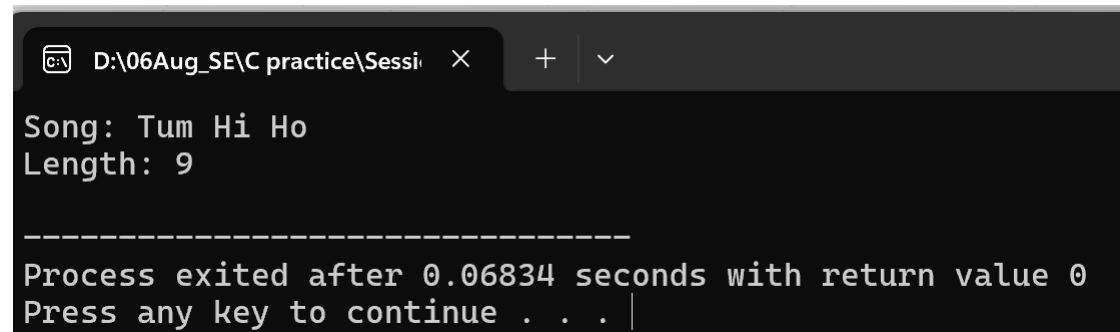
Strings Topics: Declaring and handling strings. **Common functions:** `strlen()`, `strcpy()`, `strcmp()`. **Example:** Create a username from name input. **Pending**

Tasks

1. Declare a string variable called `songTitle` and assign it the value 'Tum Hi Ho'. Print the length of the string using `strlen()`.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char songTitle[] = "Tum Hi Ho";
7
8      printf("Song: %s\n", songTitle);
9      printf("Length: %lu\n", strlen(songTitle));
10
11     return 0;
12 }
```

Output:



```
D:\06Aug_SE\C practice\Sessi  X  +  v
Song: Tum Hi Ho
Length: 9

-----
Process exited after 0.06834 seconds with return value 0
Press any key to continue . . . |
```

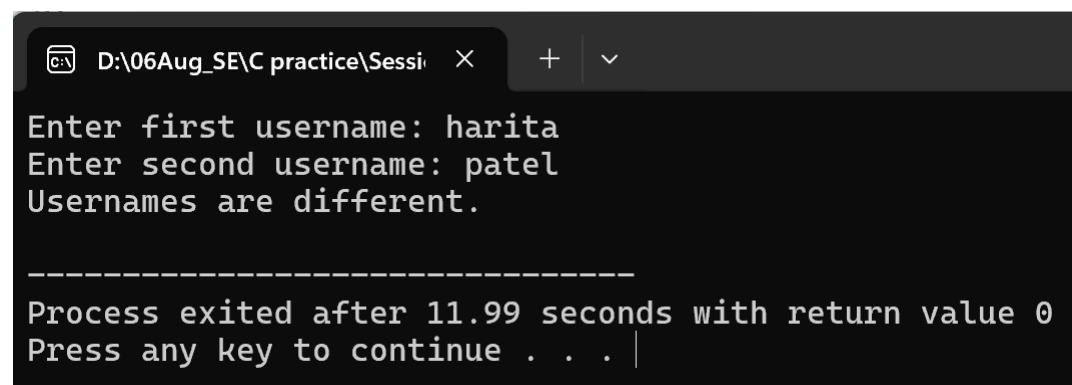
2. Take input for two usernames (as strings) and compare them using `strcmp()`. Display whether they are the same or different.

```

1  #include <stdio.h>
2  #include <string.h>
3  int main()
4  {
5      char username1[50];
6      char username2[50];
7
8      printf("Enter first username: ");
9      scanf("%49s", username1);
10
11     printf("Enter second username: ");
12     scanf("%49s", username2);
13
14     if(strcmp(username1, username2) == 0)
15     {
16         printf("Both usernames are same.\n");
17     }
18     else
19     {
20         printf("Usernames are different.\n");
21     }
22
23     return 0;
24 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi
Enter first username: harita
Enter second username: patel
Usernames are different.

-----
Process exited after 11.99 seconds with return value 0
Press any key to continue . . .

```

- Copy the string 'Flipkart' into another string variable called shoppingApp using strcpy(), then print the value of shoppingApp.

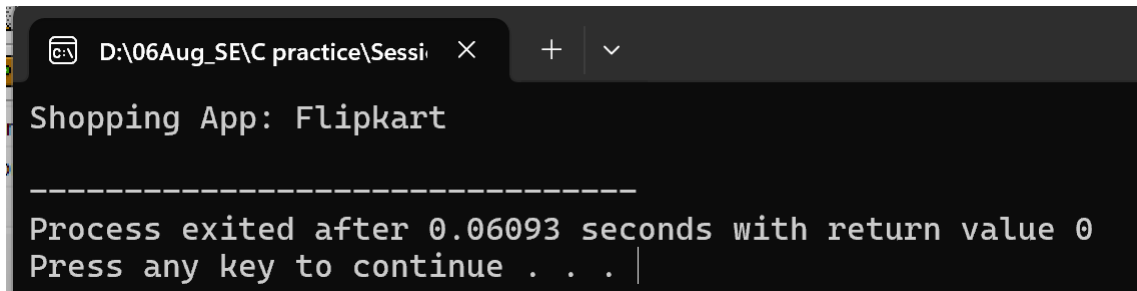
Hint: Make sure to declare enough space for the destination string.

```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char shoppingApp[20];
7      strcpy(shoppingApp, "Flipkart");
8      printf("Shopping App: %s\n", shoppingApp);
9      return 0;
10 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi  X  +  v
Shopping App: Flipkart
-----
Process exited after 0.06093 seconds with return value 0
Press any key to continue . . . |

```

4. Build a small script that takes the user's full name as input and creates a username by copying only the first 5 characters using `strcpy()`. Print the generated username.

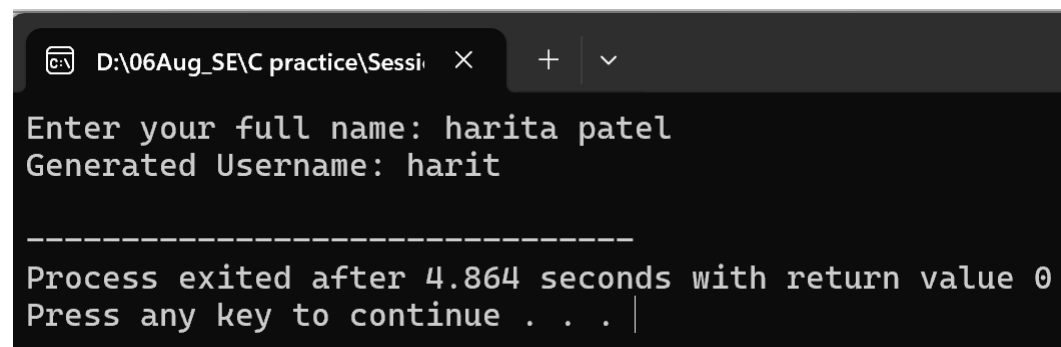
Constraint: If the name is shorter than 5 characters, use the full name as the username.

```

1  #include <stdio.h>
2  #include <string.h>
3  int main(){
4      char name[100];
5      char username[100];
6      char temp[100];
7      printf("Enter your full name: ");
8      fgets(name, sizeof(name), stdin);
9
10     name[strcspn(name, "\n")] = '\0';
11
12     if(strlen(name) <= 5){
13         strcpy(username, name);
14     }
15     else{
16         strcpy(temp, name);
17         temp[5] = '\0';
18         strcpy(username, temp);
19     }
20
21     printf("Generated Username: %s\n", username);
22     return 0;
23 }

```

Output:



The screenshot shows a terminal window with a dark background. At the top, there is a tab labeled 'D:\06Aug_SE\C practice\Sessi' with a close button. The terminal displays the following text:

```

Enter your full name: harita patel
Generated Username: harit

-----
Process exited after 4.864 seconds with return value 0
Press any key to continue . . .

```

Session-11

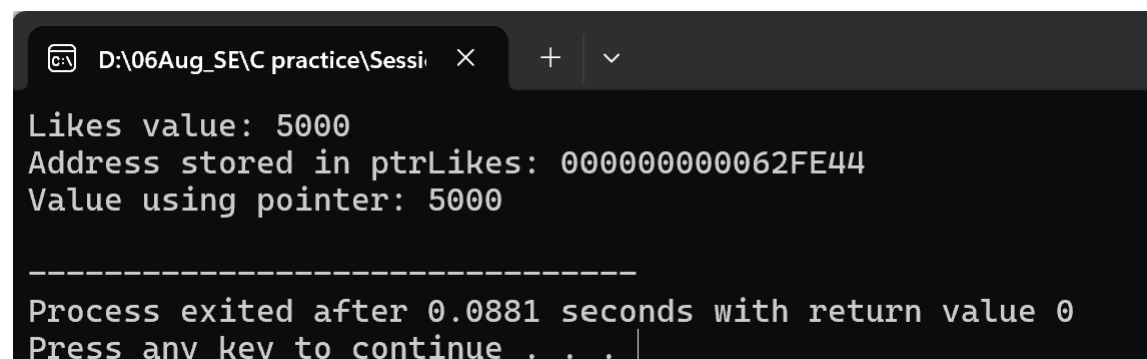
Session 11: PointersTopics:Pointer declaration, dereferencing, pointer arithmetic.Example: Swap numbers using pointers.Trainer Demo: Use memory diagram to explain address referencing.

Tasks

1. Declare an integer variable called likes and a pointer variable called ptrLikes; assign likes a value, point ptrLikes to likes, and print both the value and the address stored in ptrLikes.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int likes = 5000;
6      int *ptrLikes;
7
8      ptrLikes = &likes;
9
10     printf("Likes value: %d\n", likes);
11     printf("Address stored in ptrLikes: %p\n", (void *)ptrLikes);
12     printf("Value using pointer: %d\n", *ptrLikes);
13
14     return 0;
15 }
```

Output:



```
D:\06Aug_SE\C practice\Sessi  X  +  v
Likes value: 5000
Address stored in ptrLikes: 000000000062FE44
Value using pointer: 5000

-----
Process exited after 0.0881 seconds with return value 0
Press any key to continue . . .
```

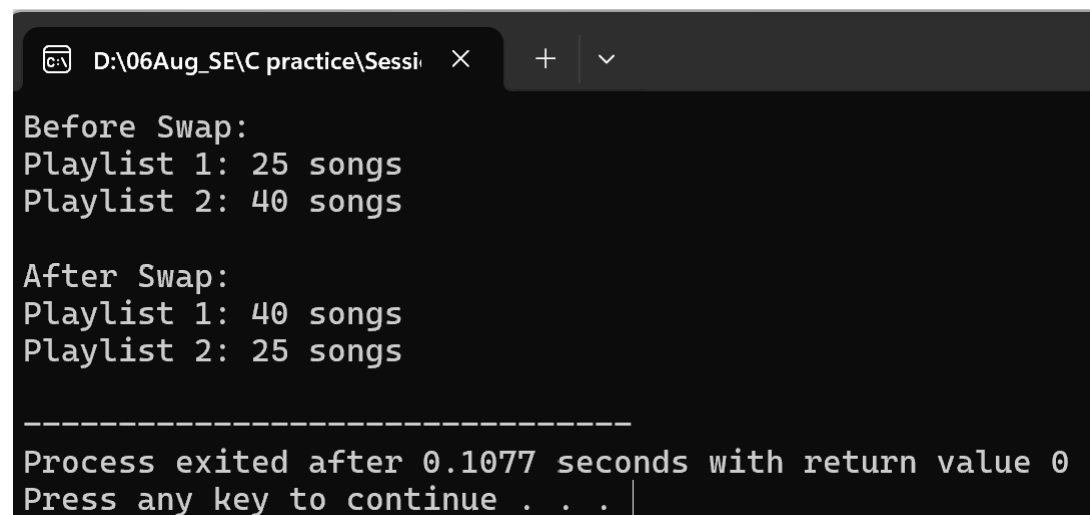
2. Write a function swapPlaylistCounts(int *a, int *b) that swaps the number of songs in two Spotify playlists using pointers, then call the function in main and print the swapped values.

```

1  #include <stdio.h>
2
3  void swapPlaylistCounts(int *a, int *b){
4      int temp;
5
6      temp = *a;
7      *a = *b;
8      *b = temp;
9  }
10 int main(){
11     int playlist1 = 25;
12     int playlist2 = 40;
13
14     printf("Before Swap:\n");
15     printf("Playlist 1: %d songs\n", playlist1);
16     printf("Playlist 2: %d songs\n", playlist2);
17
18     swapPlaylistCounts(&playlist1, &playlist2);
19
20     printf("\nAfter Swap:\n");
21     printf("Playlist 1: %d songs\n", playlist1);
22     printf("Playlist 2: %d songs\n", playlist2);
23
24     return 0;
25 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi
Before Swap:
Playlist 1: 25 songs
Playlist 2: 40 songs

After Swap:
Playlist 1: 40 songs
Playlist 2: 25 songs

-----
Process exited after 0.1077 seconds with return value 0
Press any key to continue . . .

```

- Given an array of 5 order amounts (e.g., Zomato orders), use a pointer to iterate through the array and print each amount along with its memory address.

Hint: Use pointer arithmetic to move to the next element.

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int orders[5] = {250, 180, 320, 450, 200};
6
7      int *ptr = orders;
8
9      for(int i = 0; i < 5; i++)
10     {
11         printf("Order Amount: %d\n", *ptr);
12         printf("Memory Address: %p\n\n", (void *)ptr);
13
14         ptr++;
15     }
16
17     return 0;
18 }

```

Output:

```

D:\06Aug_SE\C practice\Sessi  X  +  v
Order Amount: 250
Memory Address: 000000000062FE30

Order Amount: 180
Memory Address: 000000000062FE34

Order Amount: 320
Memory Address: 000000000062FE38

Order Amount: 450
Memory Address: 000000000062FE3C

Order Amount: 200
Memory Address: 000000000062FE40

-----
Process exited after 0.09793 seconds with return value 0
Press any key to continue . . .

```

4. Create a function `incrementFollowers(int *followers, int n)` that increases each follower count in an array (representing Instagram followers for 5 friends) by 100 using pointer arithmetic, then print the updated counts.

```

1  #include <stdio.h>
2  void incrementFollowers(int *followers, int n){
3      for(int i = 0; i < n; i++){
4          *followers = *followers + 100;
5          followers++;
6      }
7  }
8
9  int main(){
10     int followers[5] = {1200, 2500, 3400, 1800, 5000};
11     printf("Before Increment:\n");
12     for(int i = 0; i < 5; i++){
13         printf("%d\n", followers[i]);
14     }
15
16     incrementFollowers(followers, 5);
17     printf("\nAfter Increment:\n");
18
19     for(int i = 0; i < 5; i++){
20         printf("%d\n", followers[i]);
21     }
22
23     return 0;
24 }

```

Output:

```

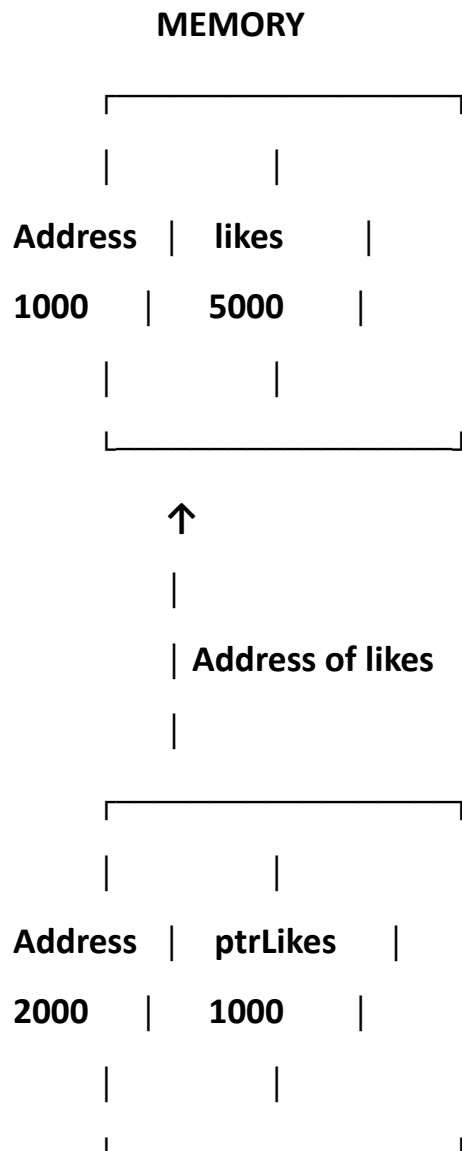
D:\06Aug_SE\C practice\Sessi  X  +  v
Before Increment:
1200
2500
3400
1800
5000

After Increment:
1300
2600
3500
1900
5100

-----
Process exited after 0.1117 seconds with return value 0
Press any key to continue . . . |

```

5. Draw a simple memory diagram (on paper or using any drawing tool) showing how a pointer references the address of a variable, using the variables from your likes and ptrLikes example; label the variable, pointer, and address.



Session-12

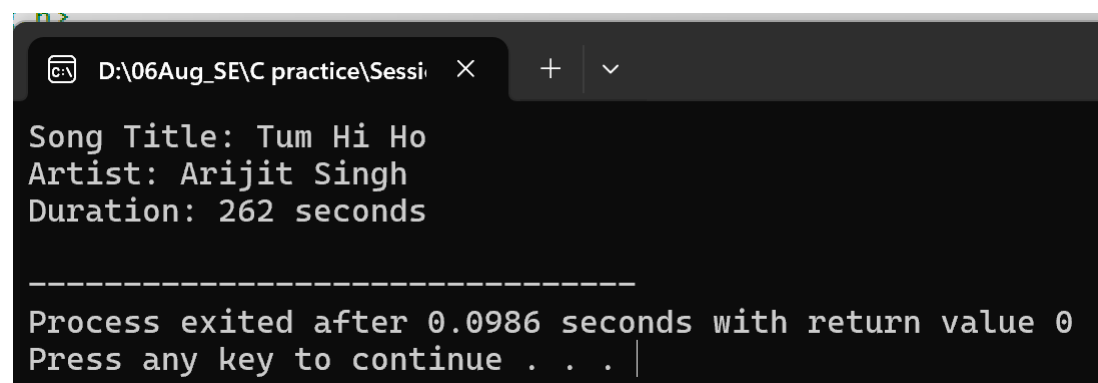
Structures Topics: Declaring, initializing structures. Nested structures. Example: Store student details (name, marks, grade). Pending

Tasks

1. Declare a structure named Playlist to store details of a song: title (string), artist (string), and duration in seconds (integer). Initialize one Playlist variable with your favorite song's details and print each field.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  struct Playlist
5  {
6      char title[100];
7      char artist[100];
8      int duration;
9  };
10
11 int main()
12 {
13     struct Playlist song = {
14         "Tum Hi Ho",
15         "Arijit Singh",
16         262
17     };
18
19     printf("Song Title: %s\n", song.title);
20     printf("Artist: %s\n", song.artist);
21     printf("Duration: %d seconds\n", song.duration);
22
23     return 0;
24 }
```

Output:



The screenshot shows a terminal window with a dark background. The title bar at the top indicates the file path 'D:\06Aug_SE\practice\Sessi'. The output of the program is displayed in a monospaced font. It shows the song title 'Tum Hi Ho', the artist 'Arijit Singh', and the duration '262 seconds'. Below this, a separator line of dashes is shown, followed by the message 'Process exited after 0.0986 seconds with return value 0' and 'Press any key to continue . . . |'.

```
D:\06Aug_SE\practice\Sessi >
Song Title: Tum Hi Ho
Artist: Arijit Singh
Duration: 262 seconds

-----
Process exited after 0.0986 seconds with return value 0
Press any key to continue . . . |
```

2. Create a structure called `FoodItem` to store Zomato-style menu data: `itemName` (string), `price` (float), and `rating` (float). Initialize an array of 3 `FoodItem` variables with real menu items and display their details using a loop.

```
1  #include <stdio.h>
2  struct FoodItem{
3      char itemName[100];
4      float price;
5      float rating;
6  };
7
8  int main(){
9      struct FoodItem menu[3] = {
10         {"Paneer Pizza", 299.00, 4.5},
11         {"Veg Biryani", 199.00, 4.3},
12         {"Masala Dosa", 149.00, 4.6}
13     };
14
15     for(int i = 0; i < 3; i++){
16         printf("\nFood Item %d\n", i + 1);
17         printf("Name: %s\n", menu[i].itemName);
18         printf("Price: %.2f\n", menu[i].price);
19         printf("Rating: %.1f\n", menu[i].rating);
20     }
21
22     return 0;
23 }
```

Output:

```
D:\06Aug_SE\C practice\Sessi X + v

Food Item 1
Name: Paneer Pizza
Price: 299.00
Rating: 4.5

Food Item 2
Name: Veg Biryani
Price: 199.00
Rating: 4.3

Food Item 3
Name: Masala Dosa
Price: 149.00
Rating: 4.6

-----
Process exited after 0.08728 seconds with return value 0
Press any key to continue . . . |
```

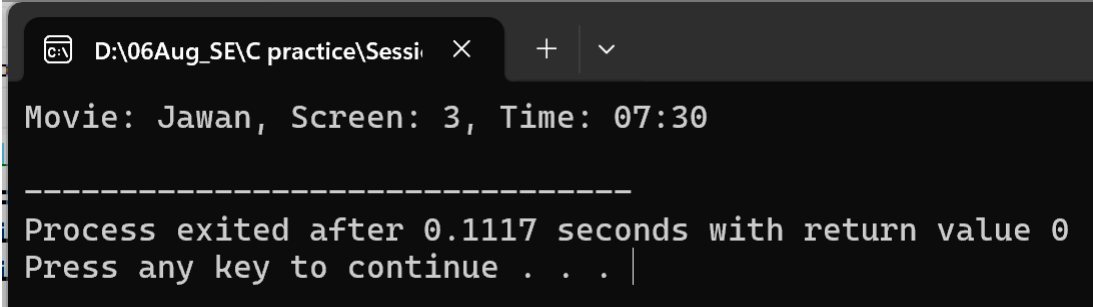
3. Define a nested structure called MovieShow for a BookMyShow-style app: Movie (string), Screen (integer), and a nested structure Time with hours and minutes (integers). Create and initialize a MovieShow variable for any movie and print its details in the format 'Movie: X, Screen: Y, Time: HH:MM'.

```

1  #include <stdio.h>
2  struct Time{
3      int hours;
4      int minutes;
5  };
6
7  struct MovieShow{
8      char movie[100];
9      int screen;
10     struct Time time;
11 };
12
13 int main(){
14     struct MovieShow show ={
15         "Jawan",
16         3,
17         {7, 30}
18     };
19     printf("Movie: %s, Screen: %d, Time: %02d:%02d\n",
20         show.movie,
21         show.screen,
22         show.time.hours,
23         show.time.minutes);
24     return 0;
25 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi X + v
Movie: Jawan, Screen: 3, Time: 07:30
-----
Process exited after 0.1117 seconds with return value 0
Press any key to continue . . . |

```

4. Build a structure called InstaProfile with fields: username (string), followers (integer), and a nested structure Bio with fields: description (string) and age (integer). Initialize an InstaProfile variable with your own details and display all fields

```

1  #include <stdio.h>
2  struct Bio{
3      char description[200];
4      int age;
5  };
6  struct InstaProfile{
7      char username[50];
8      int followers;
9      struct Bio bio;
10 };
11 int main(){
12     struct InstaProfile profile ={
13         "harita",
14         5000,
15         {"Computer Science Student", 21}
16     };
17     printf("Instagram Profile\n");
18     printf("-----\n");
19     printf("Username: %s\n", profile.username);
20     printf("Followers: %d\n", profile.followers);
21     printf("Bio: %s\n", profile.bio.description);
22     printf("Age: %d\n", profile.bio.age);
23     return 0;
24 }

```

Output:

```

D:\06Aug_SE\C practice\Sessi X + v
Instagram Profile
-----
Username: harita
Followers: 5000
Bio: Computer Science Student
Age: 21
-----
Process exited after 0.1347 seconds with return value 0
Press any key to continue . . . |

```

Session-13

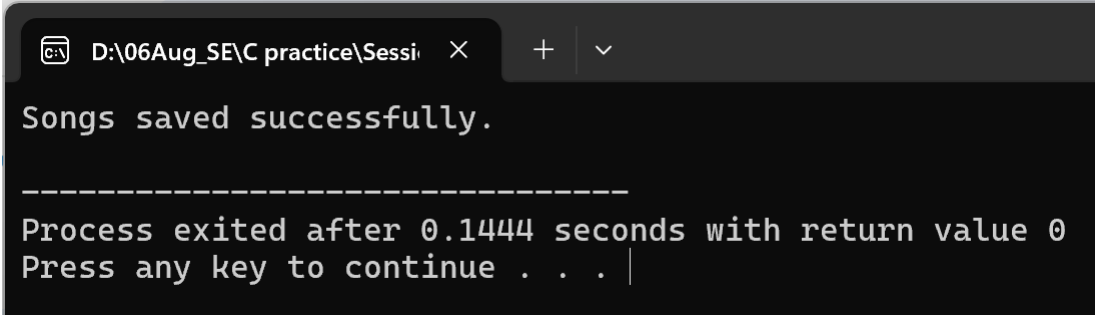
File HandlingTopics:Open, write, read, close files.Modes: r, w, a.Example:
Write and read "daily goals.txt."Pending

Tasks

1. Create a file called playlist.txt and write the names of your top 3 favorite songs from Spotify into it using write mode (w).

```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *fp;
6
7      fp = fopen("playlist.txt", "w");
8
9      if (fp == NULL)
10     {
11         printf("File could not be created.\n");
12         return 1;
13     }
14
15     fprintf(fp, "Perfect\n");
16     fprintf(fp, "Love Story\n");
17     fprintf(fp, "Shape of You\n");
18
19     fclose(fp);
20
21     printf("Songs saved successfully.\n");
22
23     return 0;
24 }
```

Output:

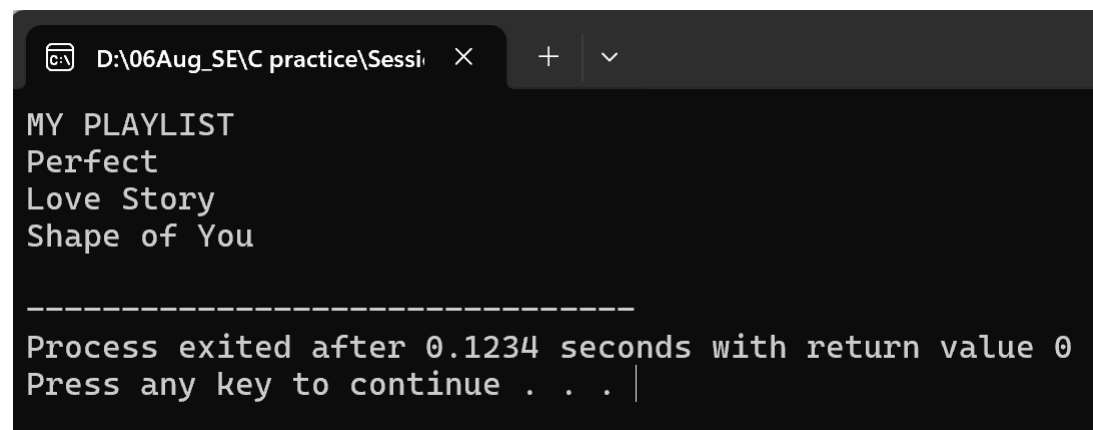


```
D:\06Aug_SE\C practice\Sessi  X  +  v
Songs saved successfully.
-----
Process exited after 0.1444 seconds with return value 0
Press any key to continue . . . |
```

2. Open `playlist.txt` in read mode (`r`) and display each song name on a separate line in the console.

```
1  #include <stdio.h>
2  int main(){
3      FILE *fp;
4      char song[100];
5
6      fp = fopen("playlist.txt", "r");
7
8      if (fp == NULL){
9          printf("File not found.\n");
10         return 1;
11     }
12
13     printf("MY PLAYLIST\n");
14
15     while (fgets(song, sizeof(song), fp) != NULL){
16         printf("%s", song);
17     }
18
19     fclose(fp);
20
21     return 0;
22 }
```

Output:



```
D:\06Aug_SE\C practice\Sessi x + v
MY PLAYLIST
Perfect
Love Story
Shape of You
-----
Process exited after 0.1234 seconds with return value 0
Press any key to continue . . . |
```

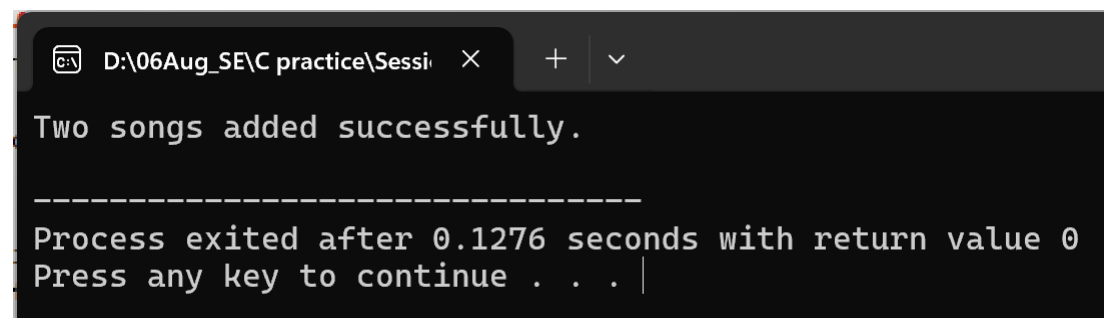
3. Add two more song names to `playlist.txt` without deleting the existing ones by opening the file in append mode (`a`).

```

1  #include <stdio.h>
2  int main(){
3      FILE *fp;
4
5      fp = fopen("playlist.txt", "a");
6
7      if (fp == NULL){
8          printf("File could not be opened.\n");
9          return 1;
10     }
11
12     fprintf(fp, "You Belong With Me\n");
13     fprintf(fp, "All of Me\n");
14
15     fclose(fp);
16
17     printf("Two songs added successfully.\n");
18
19     return 0;
20 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi  X  +  v
Two songs added successfully.
-----
Process exited after 0.1276 seconds with return value 0
Press any key to continue . . . |

```

4. Write a program that reads all song names from playlist.txt and prints only those that contain the word 'love' (case-insensitive).

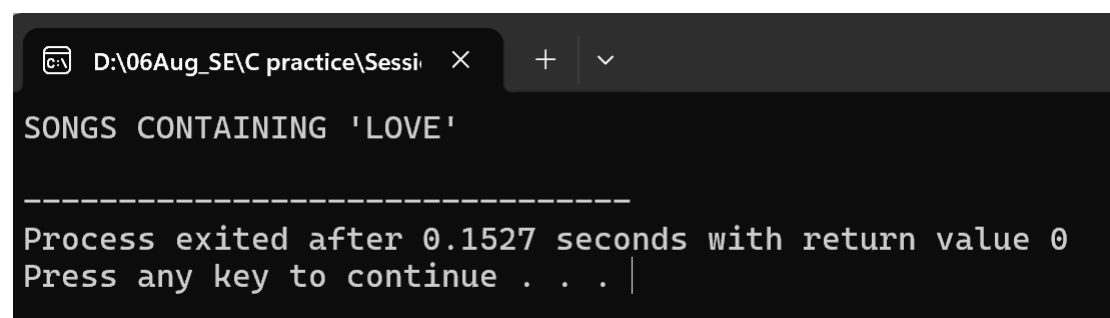
Hint: Use the 'in' keyword or equivalent string method for filtering.

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4  int main(){
5      FILE *fp;
6      char song[100];
7      char lower[100];
8
9      fp = fopen("playlist.txt", "r");
10
11     if (fp == NULL){
12         printf("File not found.\n");
13         return 1;
14     }
15
16     printf("SONGS CONTAINING 'LOVE'\n");
17
18     while (fgets(song, sizeof(song), fp) != NULL){
19         for (int i = 0; song[i] != '\0'; i++){
20             lower[i] = tolower(song[i]);
21         }
22
23         lower[i] = '\0';
24
25         if (strstr(lower, "love") != NULL){
26             printf("%s", song);
27         }
28     }
29     fclose(fp);
30     return 0;
31 }

```

Output:



```

D:\06Aug_SE\C practice\Sessi
SONGS CONTAINING 'LOVE'
-----
Process exited after 0.1527 seconds with return value 0
Press any key to continue . . . |

```

Session 14

Debugging & Best Practices Topics: Common syntax/runtime errors.

Importance of commenting & indentation. Pending

Tasks

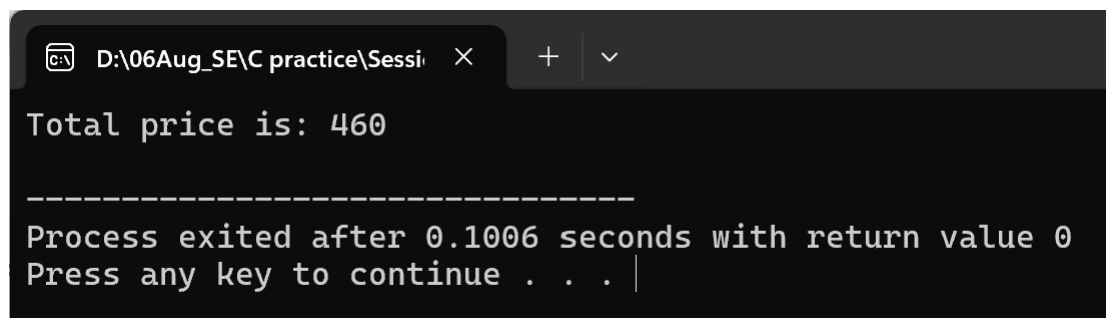
1. Given the following buggy code meant to calculate the total price of a Zomato order, identify and fix the syntax and runtime errors:

let items = ["Burger", "Pizza", "Fries"];
let prices = [120, 250, 90];
let total = 0;
for (i = 0; i < items.length; i++) {
total += prices[i];
}
console.log("Total price is: " + total);

Hint: Watch for assignment and loop variable issues.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int prices[] = {120, 250, 90};
6      int total = 0;
7      int i;
8
9      for (i = 0; i < 3; i++)
10     {
11         total += prices[i];
12     }
13
14     printf("Total price is: %d\n", total);
15
16     return 0;
17 }
```

Output:

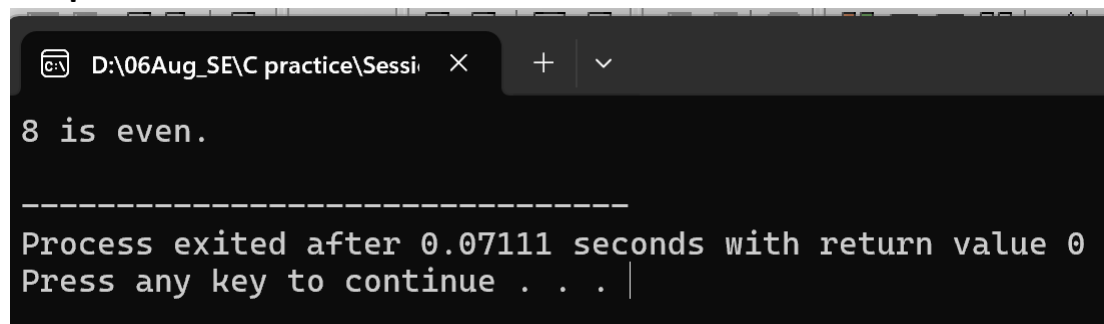


```
D:\06Aug_SE\C practice\Sessi  X  +  v
Total price is: 460
-----
Process exited after 0.1006 seconds with return value 0
Press any key to continue . . . |
```

2. Rewrite the following code to improve its indentation and add comments explaining each step, so that a beginner can understand what it does:
- ```
function isEven(num){if(num%2==0){return true;}else{return false;}}
```

```
1 #include <stdio.h>
2 int isEven(int num){
3 if (num % 2 == 0){
4 return 1;
5 }
6 else{
7 return 0;
8 }
9 }
10
11 int main(){
12 int number = 8;
13 if (isEven(number)){
14 printf("%d is even.\n", number);
15 }
16 else{
17 printf("%d is odd.\n", number);
18 }
19 return 0;
20 }
```

Output:



```
D:\06Aug_SE\C practice\Sessi X + v
8 is even.

Process exited after 0.07111 seconds with return value 0
Press any key to continue . . .
```

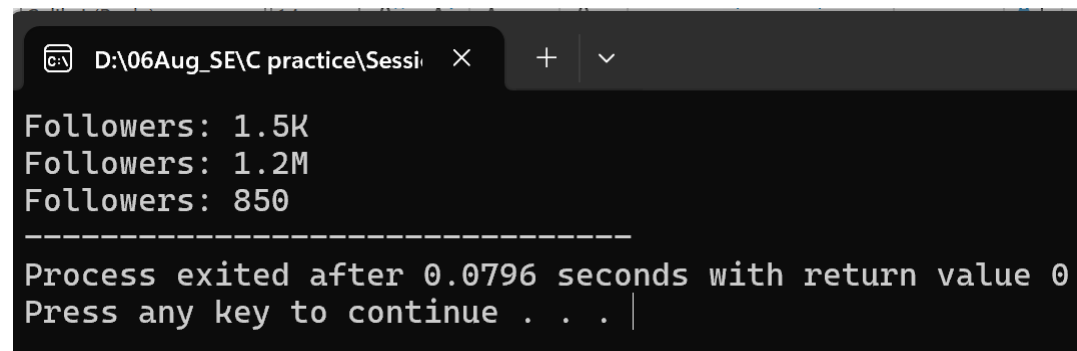
3. Write a function `formatFollowersCount(count)` that takes a number and returns a formatted string like Instagram: 1500 as '1.5K', 1200000 as '1.2M', and numbers below 1000 as-is. Add clear comments and use proper indentation.

```

1 #include <stdio.h>
2 void formatFollowersCount(int count){
3 if (count >= 1000000){
4 printf("%.1fM", count / 1000000.0);
5 }
6 else if (count >= 1000){
7 printf("%.1fK", count / 1000.0);
8 }
9 else{
10 printf("%d", count);
11 }
12 }
13
14 int main()
15 {
16 printf("Followers: ");
17 formatFollowersCount(1500);
18
19 printf("\nFollowers: ");
20 formatFollowersCount(1200000);
21
22 printf("\nFollowers: ");
23 formatFollowersCount(850);
24 return 0;
25 }

```

**Output:**



```

D:\06Aug_SE\C practice\Sessi x + v
Followers: 1.5K
Followers: 1.2M
Followers: 850

Process exited after 0.0796 seconds with return value 0
Press any key to continue . . .

```

4. You are given a piece of JavaScript code that should print all even numbers from 1 to 10, but it doesn't work as expected:  
 for (let i = 1; i <= 10; i++) {  
   if (i % 2 = 0) {  
     console.log(i)  
   }  
 }  
 Find and fix the error, then rewrite the code with good indentation and at least one comment explaining the logic.

```

1 #include <stdio.h>
2 int isEven(int num){
3 if (num % 2 == 0){
4 return 1;
5 }
6 else{
7 return 0;
8 }
9 }
10
11 int main(){
12 int number = 8;
13 if (isEven(number)){
14 printf("%d is even.\n", number);
15 }
16 else{
17 printf("%d is odd.\n", number);
18 }
19 return 0;
20 }

```

**Output:**

```

D:\06Aug_SE\C practice\Sessi X + v
8 is even.

Process exited after 0.06199 seconds with return value 0
Press any key to continue . . .

```